Entrenamiento de Redes Neuronales

(Training of Neural Networks) — Apuntes en castellano

Basado en el material: Data Mining III — Ander González Docasal, Mondragon Unibertsitatea (agonzalezdo@mondragon.edu)

---

1. Objetivo de la Red Neuronal (Goal of the Neural Network)

Dado un conjunto de datos (*dataset*) con entradas **X** y salidas **Y**, y siendo θ **(theta)** el conjunto de parámetros de la red neuronal (compuesto por los pesos **W** y los sesgos o *bias* **b**), el objetivo principal es **ajustar** dichos parámetros para que, dada una entrada individual $x_i \in X$, la salida de la red $f(x_i, \theta)$ **se aproxime** a $y_i \in Y$.

$$f(x_i, \theta) \sim y_i$$

Por ejemplo, en un problema de clasificación, la red produce una distribución de probabilidades (*Probability*) que debería parecerse a la etiqueta real (*Label*), por ejemplo, un vector *one-hot* donde solo la clase correcta vale 1.

2. Función de coste (Cost function / Loss function)

Para poder ajustar los parámetros necesitamos **calcular la similitud** entre la salida de la red y la salida deseada. La **función de pérdida o coste** (*loss function / cost function*) es una función definida positiva (*positive-definite*) que mide esta desviación:

$$L(Y, f(X, \theta))$$

- Si $f(X, \theta) \sim Y$, **entonces** $L(Y, f(X, \theta)) \sim 0$ (la red acierta, error casi nulo)
- Si $f(X, \theta)$ **no** se parece a Y, **entonces** $L(Y, f(X, \theta)) >> 0$ (la red falla, error grande)

Existen **múltiples funciones de coste**, mejores o peores según la tarea a resolver.

2.1 Regresión (Regression) — la salida es un número real

Error cuadrático medio (Mean Squared Error, MSE)

$$L(Y, P) = -(1/N) \cdot \Sigma \, ||y_i - p_i||^2$$

Error absoluto medio (Mean Absolute Error, MAE)

$$L(Y, P) = -(1/N) \cdot \Sigma \, ||y_i - p_i||$$

2.2 Clasificación (Classification) — la salida indica una clase

Entropía cruzada binaria (Binary Cross-Entropy)

$$L(Y, P) = -(1/N) \cdot \Sigma\ [y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i)] \quad \text{con } y_i \in \{0,1\}$$

Entropía cruzada multiclase (Multiclass Cross-Entropy)

$$L(Y, P) = -(1/N) \cdot \Sigma\ (y_i^{(k)} \cdot \log(p_i^{(k)}))$$

donde $y_i \in \{0, 1, \ldots, K-1\}$ y: $y_i^{(k)} = 0$ si $y_i \neq k$; $y_i^{(k)} = 1$ si $y_i = k$.

3. Descenso del gradiente (Gradient Descent)

3.1 Entrenamiento (Training)

Sabemos calcular la inferencia de una red neuronal y su rendimiento (*performance*) para un dataset dado. La pregunta es: **¿cómo entrenamos una red neuronal? ¿Cómo mejoramos su rendimiento?**

El objetivo es **minimizar la función de coste**:

$$\theta? : \min_\theta L(Y, f(X, \theta))$$

Es un **problema de optimización** (*optimisation problem*) de la función de coste, una función con *muchas* variables.

3.2 Gradiente (Gradient)

Se utiliza una técnica llamada **descenso del gradiente** (*gradient descent*) para encontrar el mínimo de la función de coste. Primero se calcula el **gradiente** de la función de coste:

$$\nabla L$$

donde, en general, el gradiente es el vector de derivadas parciales respecto a cada variable:

$$\nabla f(x_1, \ldots, x_n) = (\partial f/\partial x_1, \ldots, \partial f/\partial x_n)$$

3.3 Actualización de parámetros

En cada paso (*step*), los parámetros de la red se actualizan con un valor **proporcional al gradiente** de la función de pérdida (con signo negativo, para ir "cuesta abajo"):

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \nabla L$$

El parámetro proporcional α **(alpha)** se llama **tasa de aprendizaje** (*learning rate*).

3.4 Efecto de la tasa de aprendizaje (Effect of the learning rate)

- **Demasiado grande (too large):** los pasos son tan grandes que el algoritmo "rebota" de un lado a otro sin converger bien (oscila o incluso diverge).
- **Tamaño correcto (right size):** converge de forma suave y directa hacia el mínimo.
- **Demasiado pequeña (too small):** converge, pero muy lentamente, dando pasos muy cortos.

4. Descenso del gradiente estocástico (Stochastic Gradient Descent, SGD)

Calcular el gradiente exacto usando **todo** el dataset es muy costoso computacionalmente, y su complejidad aumenta con el tamaño del dataset **N**.

Por ello, en la práctica, el gradiente se actualiza tras calcular únicamente **B** muestras diferentes. **B** se llama **tamaño de lote** (*batch-size*).

- El *batch-size* suele ser la **primera dimensión** de los tensores de entrada y salida de la red.
- **Gradient Descent** clásico: trayectoria suave hacia el mínimo (usa todo el dataset).
- **SGD:** trayectoria más "ruidosa" o en zig-zag, porque cada actualización se basa en un batch, no en el dataset completo.

5. Inicialización de pesos (Weight initialisation)

Los pesos de la red neuronal se pueden inicializar de muchas formas:

- **Inicialización de Xavier (Xavier initialisation):** usa el tamaño de entrada y de salida (*input* + *output size*); recomendada con activaciones **tanh** o **sigmoid**.
- **Inicialización de He (He initialisation):** usa solo el tamaño de entrada (*input size only*); optimizada para **ReLU**.
- **Inicializaciones Normal y Uniforme:** dependen de la distribución de los datos de entrenamiento.

Tipo	Mejor activación	Forma distribución	Factor clave
Xavier Uniform	Tanh / Sigmoid	Plana / acotada	Entrada + salida
Xavier Normal	Tanh / Sigmoid	Campana de Gauss	Entrada + salida
He Uniform	ReLU	Plana / acotada	Solo entrada
He Normal	ReLU	Campana de Gauss	Solo entrada

6. Retropropagación (Backpropagation)

Es una técnica usada para **calcular eficientemente el gradiente** de la función de coste utilizando cálculo (*calculus*), en concreto la **regla de la cadena** (*chain rule*).

Si una red neuronal está compuesta por **L capas** (*layers*), su función de inferencia es un conjunto de funciones compuestas:

$$L(f_L(f_{L-1}(\dots , f_1(X, \theta_1), \dots , \theta_{L-1}), \theta_L), Y)$$

Aplicando la regla de la cadena, la derivada respecto a un parámetro de la primera capa es:

$$\partial L/\partial \theta_i = \partial L/\partial y \cdot \partial y/\partial h_L \cdot \partial h_L/\partial h_{L-1} \cdot \dots \cdot \partial h_2/\partial h_1 \cdot \partial h_1/\partial \theta_i$$

Se dice que **el error se propaga hacia atrás** (*backwards*) a través de todos los parámetros entrenables (*trainable parameters*) de la red — de ahí el nombre "retropropagación".

7. Partición del dataset (Dataset partition)

En machine learning, **nunca se usa todo el dataset para entrenar**. Normalmente se divide en tres subconjuntos:

- **Entrenamiento (train):** la mayoría de los datos; se usa para **actualizar los parámetros** de la red.
- **Validación (validation / dev / val):** comprueba el rendimiento del modelo **durante el entrenamiento**. El objetivo es minimizar la **pérdida de validación** (*validation loss*), no la de entrenamiento, para evitar el sobreajuste (*overfitting*).
- **Test (test / eval):** comprueba el rendimiento del modelo sobre **datos no vistos** (*unseen data*).

El **bucle de entrenamiento** (*training loop*) usa train + validación; el test queda fuera, como "datos no vistos".

Normalmente se usa entre un **70% y un 90%** de los datos para entrenamiento, repartiendo el resto entre validación y test.

- En casos con muchísimos datos (p. ej., entrenamiento de LLMs), se puede usar un **número fijo** de ejemplos para validación y test en lugar de un porcentaje, para maximizar el subconjunto de entrenamiento.
- Tanto la validación como el test deben tener una **representación clara de todas las casuísticas** que la red encontrará durante la inferencia (uso real).

8. Bucle de entrenamiento (Training loop)

- El entrenamiento se hace usando **lotes pequeños** (*small batches*) de tamaño **B**, por restricciones de memoria o eficiencia.
- En cada paso, los parámetros se actualizan usando **SGD**.
- Se llama **época** (*epoch*) al hecho de que la red recorra **todas** las muestras de train una vez.
- Después, la red recorre el subconjunto de **validación**, pero **sin actualizar los gradientes**, para calcular la pérdida de validación y comprobar el rendimiento.
- Al alcanzar un número predefinido de épocas, el bucle se detiene.

- Finalmente, el rendimiento del modelo se calcula usando el subconjunto de **test**, con las métricas estándar del problema (p. ej. matriz de confusión: TP, FP, TN, FN).

9. Sobreajuste y subajuste (Overfitting & Underfitting)

Es importante ajustar los parámetros de la red para que se adapten bien a los datos de entrenamiento. Sin embargo, **un ajuste demasiado fuerte** puede provocar que el modelo **no sea capaz de generalizar** sobre datos nuevos similares.

- **Sobreajuste (overfitting):** el modelo funciona muy bien con datos ya vistos (*seen data*), pero mal con datos no vistos (*unseen data*).
- **Subajuste (underfitting):** ocurre cuando el modelo es **demasiado simple** para capturar los matices (*nuances*) de los datos. Se soluciona aumentando la complejidad de la red o entrenando durante más tiempo.

En la gráfica de pérdida de entrenamiento (*train loss*) vs. pérdida de validación (*val loss*): al principio ambas bajan (zona de **underfitting**); llega un momento en que la pérdida de validación deja de bajar y sube, mientras la de entrenamiento sigue bajando (zona de **overfitting**). El punto óptimo entre ambas zonas se llama **punto dulce** (*sweet spot*).

10. Técnicas para combatir el sobreajuste

10.1 Dropout

Se **desactiva** una conexión entre dos neuronas con una probabilidad **p**, forzando a la red a no depender excesivamente de neuronas concretas durante el entrenamiento.

10.2 Normalización por lotes (Batch normalisation)

Técnica para **acelerar y estabilizar** el entrenamiento. **Normaliza** la salida de una capa dada para que tenga **media 0** y **varianza 1**:

$$z' = (z - \mu) / \sigma$$

Tiene **dos parámetros aprendibles** (*learnable parameters*) que se usan para **reescalar la salida** de nuevo. Se puede colocar **antes o después** de la función de activación de una capa.

10.3 Regularización (Regularisation)

Añade la contribución de los pesos a la función de coste, para **evitar parámetros muy grandes** que "eclipsen" a los más pequeños.

Regularización L1

$$L' = L + \lambda \cdot \Sigma |\theta_i|$$

Regularización L2

$$L' = L + \lambda \cdot \Sigma \theta_i^2$$

(λ, lambda, es un hiperparámetro que controla la fuerza de la regularización).

10.4 Parada temprana (Early stopping)

Cuando la pérdida de validación empieza a **aumentar** aunque la de entrenamiento siga **disminuyendo**, se recomienda **detener el entrenamiento**. Debe pararse cuando la pérdida de validación **no ha mejorado durante n épocas**; a n se le llama **paciencia** (*patience*).

11. Problemas del gradiente

11.1 Desvanecimiento del gradiente (Vanishing gradients)

Como la retropropagación actualiza los parámetros mediante **multiplicaciones** (regla de la cadena), gradientes muy pequeños pueden provocar actualizaciones **aún más pequeñas** en las primeras capas (*early layers*). Esto se llama **vanishing gradients**.

- Puede ocurrir con funciones de activación que **se saturan** en algún punto, cuyas derivadas son cercanas a 0 (p. ej. **sigmoid** o **tanh**).
- Es mejor usar **ReLU**, e incluso mejor **Leaky ReLU** o **PReLU**, ya que la derivada de ReLU es 0 para los números negativos (lo que puede causar el mismo problema en esa región).

11.2 Explosión del gradiente (Exploding gradients)

De forma similar, puede ocurrir lo contrario: un cambio drástico en algún parámetro. Se puede evitar mediante:

- **Normalizando** los datos de entrada a una distribución normal.
- Aplicando **normalización por lotes** (*batch normalisation*).
- Usando **regularización de pesos** (*weight regularisation*).
- **Recortando** (*clipping*) los gradientes a un valor máximo.

12. Ajuste de la tasa de aprendizaje (Learning rate adjustment)

Es buena práctica **disminuir el efecto de la tasa de aprendizaje** en etapas avanzadas del entrenamiento. Esto se hace con un **planificador de tasa de aprendizaje** (*Learning Rate Scheduler*).

El cambio puede ser lineal, exponencial, escalonado (*step-wise*)…, con implementaciones disponibles en la documentación de PyTorch, por ejemplo:

- **ExponentialLR**
- **LinearLR**
- **StepLR**
- (y otros)

13. Optimizadores (Optimisers)

Los **optimizadores** son variaciones del SGD que buscan **evitar el estancamiento** (*stagnation*) y **acelerar el entrenamiento**.

- **Momentum:** acelera el entrenamiento manteniendo una fracción (β, beta) de los gradientes anteriores.
- **RMSProp** (*Root Mean Square Propagation*): normaliza el gradiente usando un historial de gradientes al cuadrado anteriores, reduciendo el efecto de las iteraciones más antiguas.
- **Adam** (*Adaptive Moment Estimation*): combina ambos enfoques (momentum + RMSProp) en un único optimizador, con parámetros β**1** y β**2**.

14. Parámetros vs. Hiperparámetros (Parameters vs Hyperparameters)

- **Parámetros (parameters):** los **pesos entrenables** de la red neuronal (W y b), que se ajustan automáticamente durante el entrenamiento.
- **Hiperparámetros (hyperparameters):** valores que **determinan la arquitectura o el comportamiento** del modelo o del entrenamiento, y que se fijan de antemano (no se aprenden mediante gradiente).

Ejemplos de hiperparámetros:

- Tasa de aprendizaje (*learning rate*)
- Tamaño de lote (*batch size*)
- Número de épocas (*number of epochs*)
- Momentum
- Probabilidades de dropout
- Parámetros de regularización
- (y otros)

Resumen visual del proceso de entrenamiento

1. Se define la **función de coste** adecuada al problema (regresión → MSE/MAE; clasificación → cross-entropy binaria o multiclase).
2. Se calcula el **gradiente** de dicha función respecto a los parámetros, mediante **backpropagation**.
3. Se actualizan los parámetros con **SGD** (u optimizadores como Momentum, RMSProp o Adam), usando una **tasa de aprendizaje** adecuada (ni muy grande ni muy pequeña).
4. Se entrena por **épocas**, usando **train** para ajustar y **validación** para vigilar el sobreajuste.

5. Se aplican técnicas de regularización si hace falta: **dropout, batch normalisation, L1/L2, early stopping**.

6. Se vigilan problemas de gradiente: **vanishing** y **exploding gradients**.

7. Se evalúa el modelo final con el subconjunto de **test**, sobre datos nunca vistos.